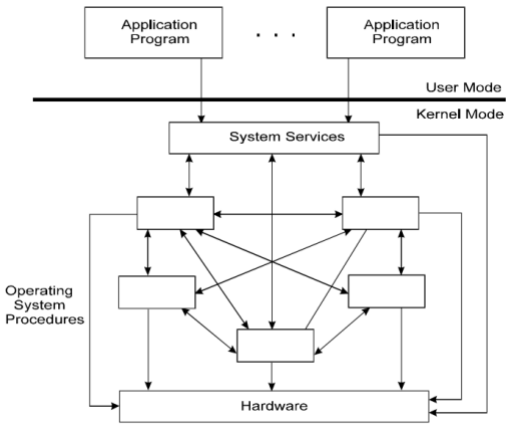
Principali architetture di Sistemi Operativi

L'architettura di un S.O. definisce la struttura interna e l'organizzazione dei suoi componenti principali, andando quindi a definire come sono progettati e interconnessi i moduli che gestiscono le diverse risorse del sistema, sia hardware che software.

Architettura monolitica

In questo modello il **Kernel**, ovvero il nucleo del S.O. (che è sempre presente in memoria RAM), detiene tutte le funzioni. Questo significa che tutte le operazioni del Sistema Operativo vengono eseguite in kernel mode, che ricordiamo essere una modalità privilegiata di esecuzione delle istruzioni da parte della CPU.



Questo non significa che il Kernel sia formato da un unico grosso pezzo di codice sempre in esecuzione, ma può tranquillamente essere suddiviso in moduli; non necessariamente tutti verranno caricati in memoria centrale all'avvio del computer, ma alcuni di essi potrebbero essere caricati in seguito, in maniera dinamica. Tuttavia, una volta terminata la fase di bootstrap, possiamo considerare di avere un unico programma in esecuzione.

Possiamo vedere facilmente sia i vantaggi che gli svantaggi di una architettura di questo tipo.

Tra i vantaggi spicca l'estrema semplicità nell'implementazione di questa architettura, in quanto ogni funzionalità del S.O. è racchiusa nel Kernel e, pertanto, ha piena di visibilità di tutte le altre funzioni presenti, che può richiamare molto facilmente. Inoltre, questa semplicità si traduce anche in una esecuzione più rapida delle istruzioni.

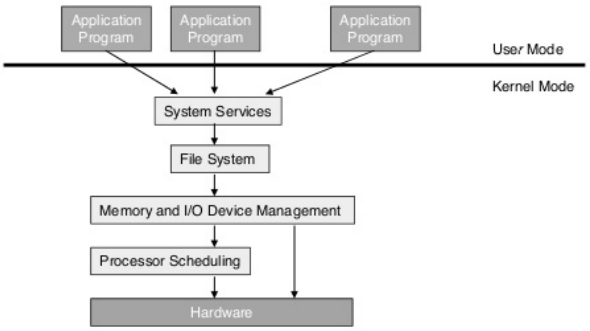
D'altro canto, questo tipo di architettura soffre di una serie di problemi legati a questo eccessivo legame che si ha tra le diverse funzionalità.

Infatti, l'aggiunta di una nuova funzione oppure la risoluzione di bug presenti comporta necessariamente la ricompilazione dell'intero Kernel; inoltre, dal momento che, potenzialmente, le funzionalità sono molto dipendenti le une con le altre, l'attività di manutenzione diventa notevolmente più complicata, ed errori presenti in un modulo possono ripercuotersi a catena su tutti gli altri. Un altro problema sta nel fatto che la dimensione del Kernel del S.O., con l'aggiungersi di nuove funzionalità, cresce sempre di più.

La maggior parte dei S.O. moderni ha avuto un passato in cui possedeva un'architettura monolitica; alcuni hanno mirato verso un'architettura di diverso tipo, come Windows e macOS, altri, come Linux ed il suo derivato Android, l'hanno mantenuta. Questo non significa che quest'ultimi debbano essere considerati obsoleti (anzi è proprio il contrario, dato l'elevatissima diffusione di entrambi), perché nel corso degli anni sono state adottate alcune caratteristiche architetturali che hanno esaltato i punti di forza di questo modello (efficienza, semplicità, velocità) e ne hanno massimizzato la stabilità, che è una delle sue principali debolezze.

Architettura a livelli (layered)

Come è facile intuire dal nome, questo tipo di architettura suddivide le funzionalità del S.O. in livelli; ciascun livello ospita funzionalità simili tra loro che, tipicamente, sono molto interconnesse. Il livello più in basso corrisponde all'hardware della macchina.



In questa architettura un livello può richiedere supporto solamente dai livelli sottostanti e può fornire servizi solo a quelli soprastanti; in genere ogni livello si interfaccia solo con quelli immediatamente inferiore e superiore.

La suddivisione in livelli semplifica notevolmente l'attività di manutenzione, aggiornamento e rilevazione di bug, oltre ad evitare che un errore si espanda verso altri moduli: è presente una netta separazione tra i diversi layer.

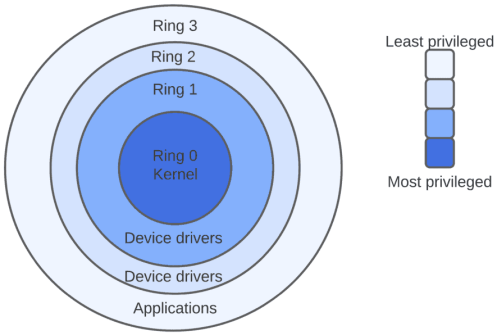
La sfida che si incontra con un'architettura di questo tipo riguarda il posizionamento di una certa funzionalità, in quanto occorre tenere in considerazione quanto detto prima, ovvero che ogni livello può fare affidamento solamente su quelli sottostanti per svolgere una determinata operazione.

Un altro svantaggio è presente nel momento in cui un livello deve richiedere un servizio (una funzionalità) offerta da un livello che non è quello immediatamente inferiore: infatti, non potrà richiamarlo direttamente, ma dovrà affidarsi ad una funzionalità presente nel livello sottostante, che a sua volta richiamerà il servizio indicato (se è al livello sottostante, altrimenti farà la stessa cosa). Questo ha un impatto negativo sulle prestazioni, in quanto si ha un **overhead** (aumento di "costi" in termini computazionali).

Architettura Onion Skin

Questo modello è una variante di quello appena descritto, in cui ogni livello (detto **ring**) è separato dagli altri tramite un meccanismo supportato a livello hardware dalla CPU (oppure simulato attraverso software); questo meccanismo prevede che ogni volta che deve essere invocato un servizio presente in un ring diverso, venga chiamata una particolare system call.

I processori della famiglia x86 (tutti gli Intel attuali) supportano 4 ring.



Questa architettura ricorda la struttura a strati di una cipolla, per questo viene chiamata Onion Skin.

Questo modello fornisce maggiori sicurezza e solidità, ma ha un prezzo in termini di computazione, in quanto invocare servizi tramite system call non è proprio economico; ricordiamo che lo stato di un processo sospeso a causa della chiamata ad una system call deve essere salvato per poter poi riprendere la sua esecuzione (al termine della system call) dal punto in cui era stato interrotto.

Architettura a microkernel

Con questa architettura stiamo parlando sempre di sistemi a livelli (layered), con la differenza che non tutti i livelli sono presenti a livello Kernel, ma alcuni possono essere spostati come moduli esterni a livello User (come se fossero dei "server" che offrono servizi).

Non esiste uno schema unico, in quanto non è presente una "regola" che indica quali livelli rimuovere dal kernel.

Qui possiamo già indicare un vantaggio, indicato anche nel nome di questo tipo di architettura: le dimensioni del Kernel sono notevolmente ridotte, perché si effettua una selezione su quali moduli mantenere a livello Kernel e quali rimuovere e predisporre a livello User. Questo si traduce in una semplificazione sia nell'aggiunta di nuove funzionalità, che possono essere inserite come moduli esterni, senza andare a modificare il Kernel (cosa che richiederebbe un aggiornamento di tutti i sistemi), sia nei controlli di correttezza effettuati al Kernel stesso, in quanto molto più compatto.

Inoltre, è meno probabile che un errore si presenti a livello di Kernel (cosa che porterebbe ad un crash dell'intero sistema); errori, e quindi crash, a livello dei moduli esterni vengono gestiti nello stesso identico modo con cui si gestirebbero quelli riscontrati in un normale programma applicativo, ovvero riavviandoli.

Ultimo vantaggio, ma non per importanza: i servizi forniti dai moduli a livello User possono essere invocati dai programmi applicativi senza richiamare system call.

Questo tipo di architettura presenta un solo svantaggio, ma molto pesante: le prestazioni sono più basse. Questo perché se funzioni presenti in moduli diversi, ma entrambi nel Kernel, possono chiamarsi direttamente, questo non può avvenire tra due funzioni presenti in moduli a livello User.

Questo viene fatto per garantire il principio di protezione tra processi, per il quale un processo può accedere solamente allo spazio di memoria a lui dedicato; pertanto, se due moduli a livello User devono comunicare tra di loro, è necessario che intervenga il S.O., con una serie di chiamate a funzione e, quindi, maggiore overhead dovuto a continui passaggi da User Mode a Kernel Mode.

Architettura ibrida

Questo tipo di architettura è una via di mezzo tra quella monolitica e quella a microkernel.